

Framework for System Simulation

Thesis – Design and Implementation

Florian Frech, 20.10.2025

1. FEM Pendulum Model with Wall Contact

- Gap Distance Calculation for Curved Contact Facets
- Impact of Time Step Size and Contact Stiffness on Energy Conservation
- Pendulum: Torque Application via Rotation Edge
- Pendulum: Gap distance dependent time step adaption

2. Heterogeneous Pendulum Model Integration (FMU/OpenSim/FEM)

- Model State Synchronization
- No Contact: Time Dependent Model Switching (*with Demo*)
- Contact: Angular Position Dependent Model Switching (*with Demo*)

3. Update Framework Architecture & Implementation Overview

- Core: Ports, CoSim, Component, FMUComponent
- System: Connections and System Setup for Execution
- Visualization of the System Graph (*with Demo*)

4. User Interaction & Guidance

- Overview Simulation Workflow
- Jupyter Widgets Example for Model Selection (*with Demo*)

5. Update Requirement Coverage

- Simulation & Modeling
- Simulation
- Heterogeneity
- User Interaction

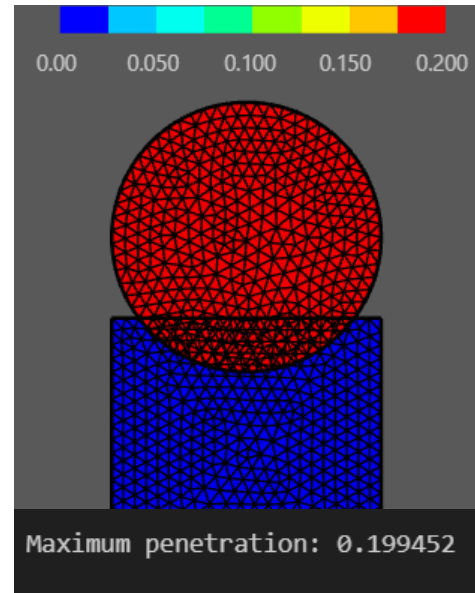
6. Next Steps

- Framework Core Expansion
- Simulation Engine & Computational Causality
- User Interaction & System Assembly
- Framework Integration & Validation

Gap Distance Calculation for Curved Contact Facet:

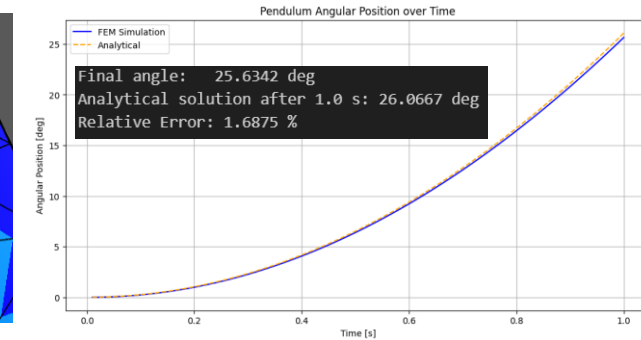
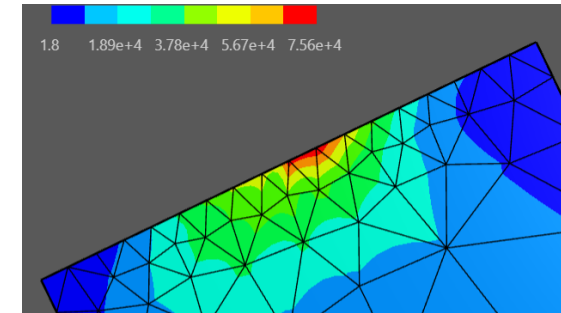
- Evaluating the gap function at multiple integration points

 - Loop over all master boundary (square) elements
 - Get element transformation object (reference element coordinates to global coordinates)
 - Get the integration rule that provides sample points within each boundary element (order = 10)
 - Map each integration point to global coordinates and evaluate the gap function (contact.gap) to get normal distance
 - Collect all gap values and find the minimum



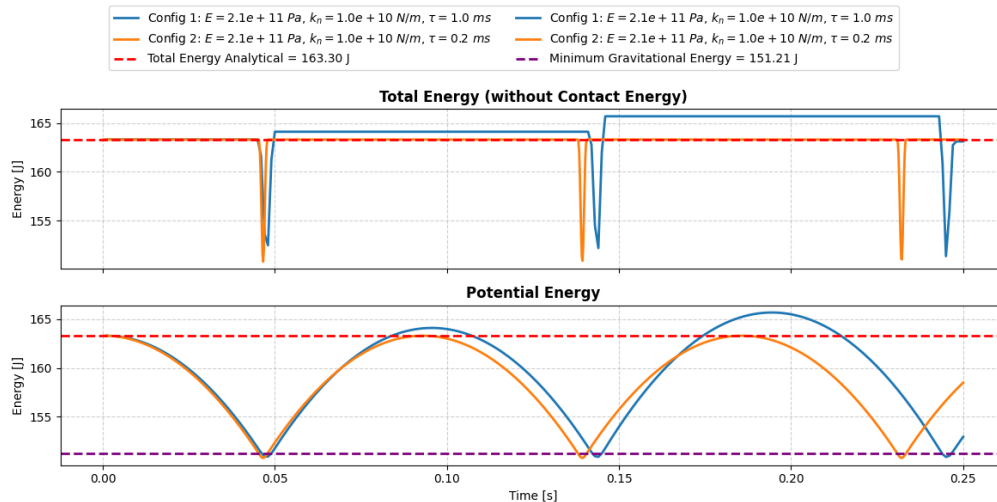
Pendulum: Torque Application via Rotation Edge

- Idea.** Apply a symmetric traction force pair left and right to the rotation point on the rotation edge that creates only a net moment (no net force)
- Smooth Heaviside function is used to **split the rotation edges into right and left halves**
- Z-component of the cross product ($\vec{r} \times \vec{n}_{rot}$) gives the **moment arm**
- Traction Force Vector:** applied in **normal direction to the rotation edge**



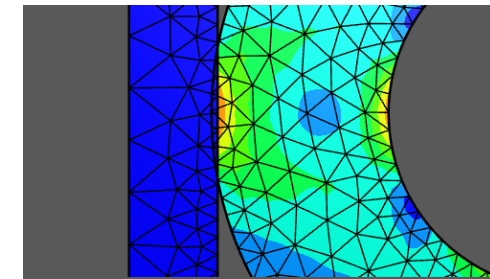
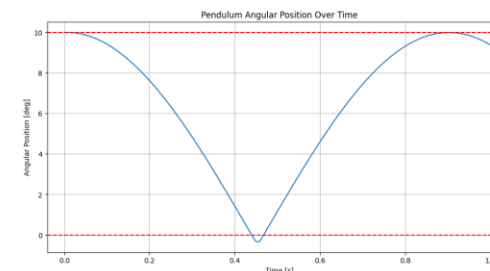
Impact of Time Stepping and Contact Stiffness on Energy Conservation:

- Incremental gap formulation depends on difference between current and previous displacements



Pendulum: Gap distance dependent time step adaption

- Time step tau is introduced as parameter in bilinear form to allow adaptive setting
- Gap distance is tracked in simulation loop and reduced for small distances to ensure energy conservation
- Allows larger time steps away from contact and high time step resolution during contact



Simulation Diagnostics

Time: t / 1.0 s	0.44739999999999994
Time step: tau in s	0.0001
Theta: theta in deg	-0.25153995917587063
Omega: omega in rad/s	-0.4675948530667177
Torque: Mz in Nm	0
Gap: d in m	-0.003510742892336417

Objective:

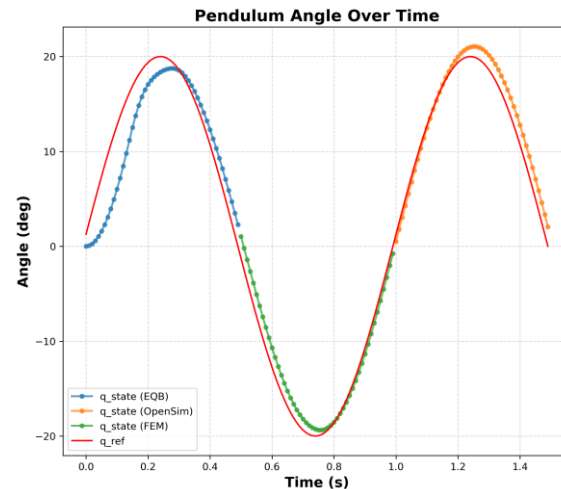
- Demonstrate that different subsystem models (FMU/OpenSim/FEM) for the pendulum can be
 - **Synchronized**
 - **Interchanged**
 - **Consistently executed**
- Synchronization is achieved **by state reinitialization**
- **Combine subsystem models for the same physical object within a MultiComponent**
 - Defines switching and synchronization rules
 - Allows state-event based switching
 - Choose model based on research question
 - Parallel execution computationally expensive

Model State Synchronization

- Pendulum state needs to be synchronized when switching between FMU/OpenSim/FEM
- Common state for all pendulum models: current angular position and angular velocity
- **Equation-based FMU:**
 - Resets the current FMU2Slave instance, sets parameters for initial values to the given state
 - Reinitializes the FMU with the new initial values and current time
- **OpenSim:**
 - updates internal state's time and set position and speed for joint coordinate
 - Model realizes for joint coordinate new given position and speed value
 - Manger object is refreshed with new model (positions) and current time (state)
- **FEM:**
 - set state method applies given angular position and angular velocity to the corresponding grid functions

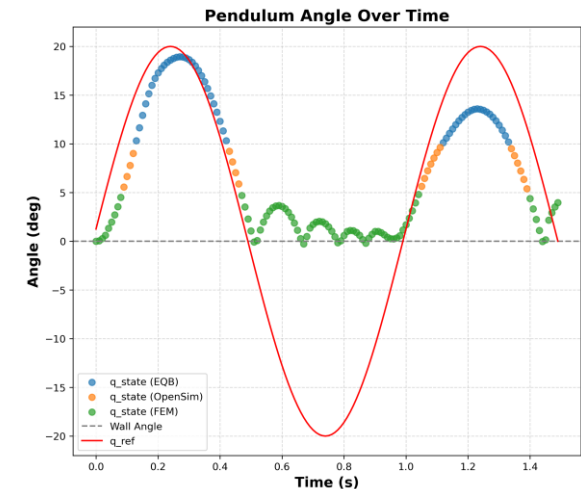
Time Event-Based Model Switch

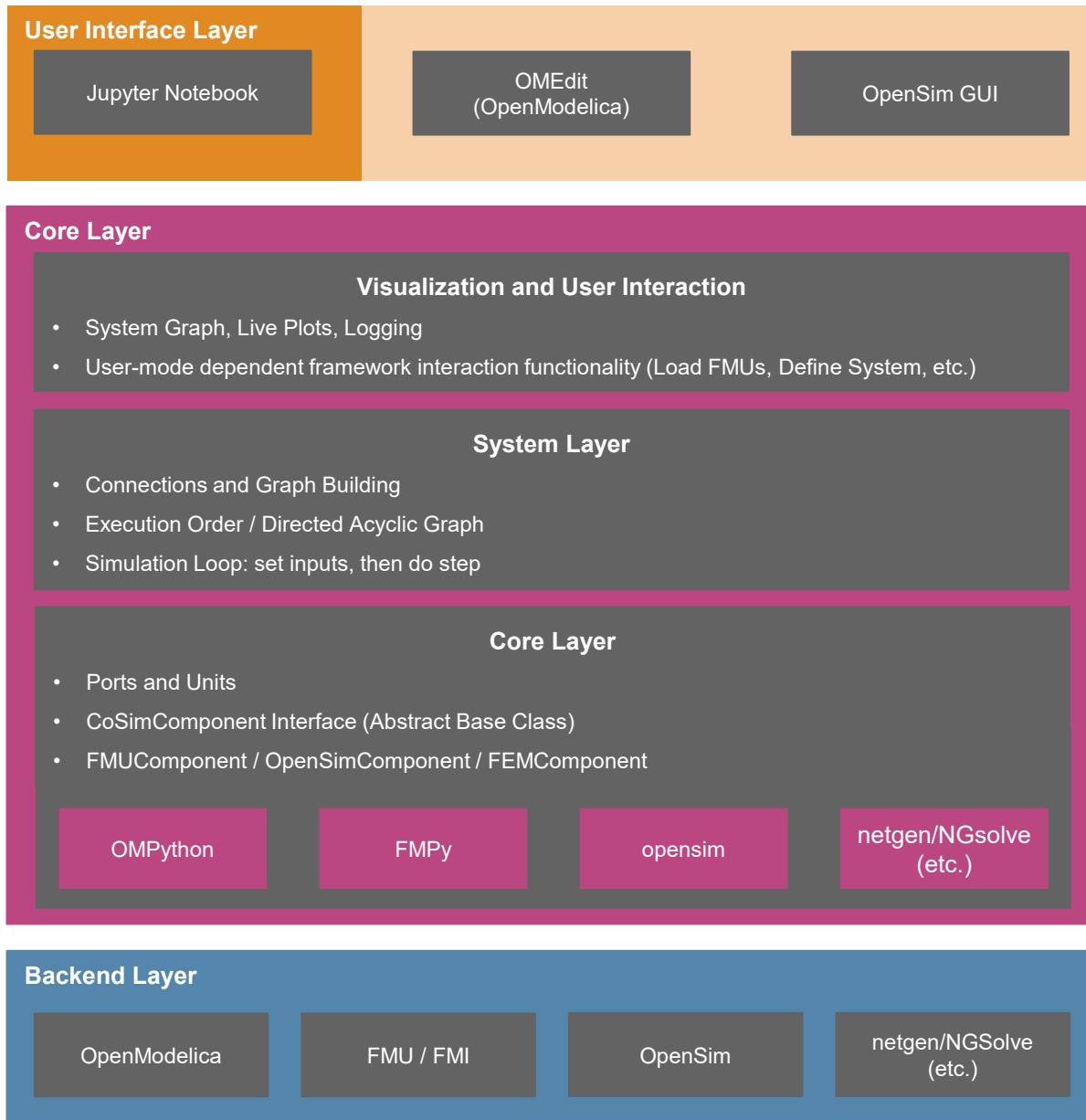
- Driven pendulum swing without wall contact
- Showcase for time dependent model switching and state synchronization
- Starts with equation-based FMU
- Switches to FEM and then to OpenSim
- Graph shows proper synchronization between models
- No discontinuities are introduced



State Event-Based Model Switching

- When the pendulum approaches the wall, the model switches first to OpenSim, and then to the FEM contact model
- Switching based on angular position of the pendulum
- Shows that the pendulum cannot follow the reference trajectory due to wall impact
- If distance threshold is reached, switch back for faster simulation





Core Layer: (implemented)

- **Ports**: PortSpecification, PortState, and PortType
 - Unified, type-safe I/O interface (PortSpec, PortState)
 - Supports REAL, INT, BOOL, STRING with unit validation via pint
- **CoSimComponent Interface**
 - Abstract base defining consistent lifecycle (initialize(), do_step(), set_inputs(), etc.)
 - Enables consistent handling of FMU, OpenSim, and FEM components
- **FMUComponent**
 - Fully functional wrapper using FMPy
 - Automatically extracts ports and units from FMU model description
 - Supports parameter initialization

System Layer: (implemented)

- **Connection Handling**
 - Defines typed links between component ports via connection objects
 - Performs port type and unit validation, detects invalid and duplicate connections
- **System Assembly**
 - Builds a directed multi-graph of modules and connections
 - Computes execution order automatically via topological sort of zero-delay dependencies
 - Supports detection of algebraic loops and parallelizable generations (CoSimComponents that do not depend on each other and can perform step in parallel)
- **Simulation Loop**
 - Generation-based executions (set inputs, then do step)
 - Propagation of output signals to destination components

Visualization and User Interaction

- **System Graph Visualization** (implemented)
 - Shows subsystems, ports, and connections
 - Indicates execution order
- **User Interaction** (prototype)
 - Create system, select models, instantiate models, connect and configure, simulate, compare

SysSimX/core/port.py:

- **PortSpec: Immutable Port Specification**
 - Name and type (REAL, INT, BOOL, and STRING)
 - Direction: Input ('in') and Output ('out')
 - Optional Unit for REAL Port Type
- **PortState: Mutable Port Runtime State**
 - Holds current value with type validation
 - tracks when the value was last updated
 - Maintains a complete history record of (time, value) pairs
- Ensures type safety and validation
- Unit enforcement: only REAL ports can have units
- Compatibility check between connected ports are validated via type/unit

SysSimX/core/base.py:

- CoSimComponent serves as the abstract base class for the co-simulation framework and defines a unified interface for integrating different simulation tools and models into a single co-simulation environment
- Metadata: name, label (for system graph), group (for clustering in graph)
- Port Management: Port Specifications and Port States
- **Abstract Lifecycle Methods:**
 - **Initialization:** Sets up component internal state and prepares it for simulation
 - **Simulation Step:** Advances the component's state by one time step
 - **State Management:** Enables saving / setting state for checkpoints, synchronization, etc.
 - **Input Handling:** Validates input port existence and sets values with optional time stamps
 - **Output Retrieval:** Returns all current outputs values

SysSimX/core/fmu_comp.py:

- Implements an FMU wrapper that integrates FMI Co-Simulation models into the co-simulation framework (bridge between standardized CoSimComponent and actual FMU instance)
- Loads and validates FMU files and extracts metadata for port and parameter setup
- **Automatic Port Discovery:**
 - Automatically creates port specification from FMU model description
 - Maps FMU variables to typed ports and preserves unit information
- **Value Reference Caching:**
 - Pre-computes FMU value references for efficient batch operations
 - No need for runtime lookups in model description during simulation
- **Batch I/O operations:**
 - Groups inputs data type for efficient FMU API calls
 - FMU API calls are only called once per type
 - Handles unit conversion for REAL ports (only magnitude of pint.quantity is set)
 - Validates input types against port specification
- **Lifecycle Management:**
 - Follows FMI initialization protocol: create instance, instantiate, setup experiment time, enter initialization mode, set parameters, and exit initialization mode
 - Sets up port states and caches for runtime efficiency
- **Simulation Stepping:**
 - Advances FMU simulation by one communication step
 - Updates output port states with new values
- **Unit-Aware Output Handling:**
 - Retrieves output in batches by type
 - Automatically attaches units to REAL quantities
 - Updates port states with time stamps

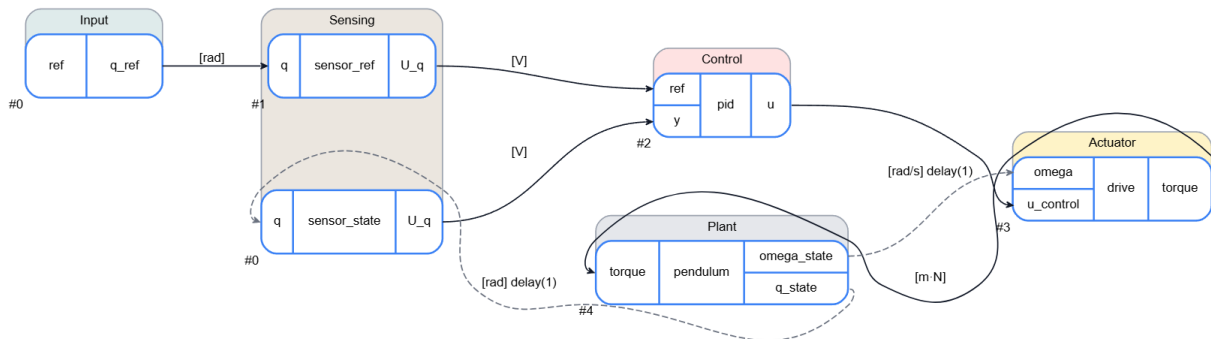
SysSimX/system/connection.py:

- Connection represent data flow links between components in a co-simulation system
- Defines a directed data flow from one component's output port to another component' input port
- **Delay:** controls the temporal coupling between components
 - **delay == 0: synchronous communication within the time step**
 - creates strict execution order (source before dest)
 - Creates algebraic coupling
 - **delay == 1: asynchronous communication to next time step**
 - No execution order constraints
 - break algebraic loops

SysSimX/viz/sysgraph.py:

- Port-based component visualization
- Connection between ports with unit labels and delay information
- Shows execution order number to help understand component scheduling and dependencies

Controlled Pendulum System



SysSimX/system/system.py:

- **Component Management**
 - Maintains registry of all simulation components and their interconnections
 - Optional grouping for organization
- **Connection Validation**
 - Components must exist before connection
 - Ports must be correctly typed (from output port to input port)
 - Type and unit compatibility between ports must be ensured
 - No duplicate connections or multiple feeding allowed
- **Graph-Based Dependency Management**
 - Full graph: complete system topology including delayed connection
 - Directed Acyclic Graph (DAG): Only zero-delay connections for execution ordering
- **Execution Order Computation**
 - Builds DAG only with zero-delay connections and checks for algebraic loops
 - Topological ordering: ensures the dependencies are satisfied before execution
 - Parallel Generations: Components in the same generation can execute concurrently
- **Simulation Execution:**
 - All components see the same state of the system
 - Components in same generation can run concurrently
 - Delayed value retrieval enables by port history
 - Multiple drivers for single port are detected

Objective:

- Provide different system interaction possibilities based the selected **user mode**
- Standard User:
 - Keep entry barrier as low as possible
 - Wrap user interaction in a Jupyter notebook (light coding and widgets interaction)
- Expert User:
 - Keep full flexibility for advanced users
 - Allow direct access of SysSimX classes, methods and functionality
- Provide **visual navigator** to support users in in the workflow (from model selection to analysis)
- Provide **templates and documentation** to leader framework user through the simulation workflow
- Save **system configurations** (FMU paths, connections, parameters, etc.) to a configuration file for later re-loading or comparison purpose

Guided Workflow

1. **System Creation:** Create a new simulation system
2. **Model Selection:** Select models for subsystems and instantiate them as CoSimComponent
3. **Connection Management:** Connect the model instances via their port names
4. **Parameter Setup:** Edit model, system, and simulation parameters
5. **System Visualization:** Generate system graph and show the execution order
6. **Execution:** Run the configured and validated system simulation (with live output)
7. **Post-Processing:** Visualize states and port histories

Expert User:

- Create the models for subsystems (equation-based) Modelica models, OpenSim, FEM, or own CoSimComponent
- Defines and configures the internal behavior (private methods, attributes, and parameter)
- Defines which functionality should be made accessible for standard users (which parameters are allowed to be configured)
- Defines which values for parameters can be selected (limits, predefined selections, etc.)
- Direct interaction with the framework through imports of classes and functionality

```
Demo 2: Controlled Pendulum
Demo 2.8: Update SysSimX

• Connections
• System
• Resolving Computational Causality
• System Graph Creation

import sys
import numpy as np
from path import Path

repo_root = Path.getcwd().parent.parent
sys.path.insert(0, str(repo_root))

Generate FMUComponents

from SysSimX.utilities.update_fmu import get_fmu_paths
from SysSimX.core.fmu_comp import FMUComponent

demo_dir_path = Path(repo_root / 'demos' / 'ControlledPendulum')
package_path = Path(demo_dir_path / 'ControlledPendulum')
fmu_output_dir = Path(demo_dir_path / 'FMUs')
fmu_paths = get_fmu_paths(package_path, fmu_output_dir, force_rebuild=False)

ref = FMUComponent(name='ref', fmu_path=fmu_paths['Reference'], group='Input')
sensor_ref = FMUComponent(name='sensor_ref', fmu_path=fmu_paths['AngleEncoder'], group='Sensing')
sensor_state = FMUComponent(name='sensor_state', fmu_path=fmu_paths['AngleEncoder'], group='Sensing')
pid = FMUComponent(name='pid', fmu_path=fmu_paths['PID.Continuous'], group='Control')
drive = FMUComponent(name='drive', fmu_path=fmu_paths['Drive'], group='Actuator')
pendulum = FMUComponent(name='pendulum', fmu_path=fmu_paths['Pendulum'], group='Plant')

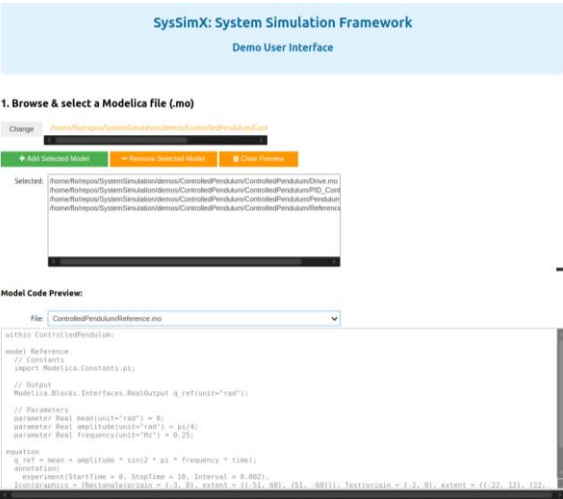
All FMUs for package 'ControlledPendulum' already exist. Skipping generation.

Change unit of input ports for PID controller to match unit of input signals

Define Connections
```

Standard User:

- Low barrier entrance for framework usage by minimizing own coding and scripting load
- Interacts mainly via GUI (ipywidgets)
- Requires sufficient documentation and guidance through simulation procedure
- Limited ability to change internal model and system parameters
- Voila provides interaction with Jupyter Notebook without code cells, interaction solely via widgets



7. Requirement Coverage: Modeling

ID	User Requirement	Status	Justification	Further Improvements
UR-1	The framework shall support the component-based modeling of multi-domain systems .	Fulfilled	- Controlled Pendulum Demo with separate subsystems - Modelica packages allow modular composition Multi-domain aspects covered (mechanical + electrical + control)	Define generic interfaces for plug-and-play across domains (implemented)
UR-2	The framework shall support the use of units for all physical quantities .	Fulfilled	Modelica subsystems use SI units consistently (checked by OpenModelica compiler) - FMUs expose units in modelDescription.xml OpenSim API provides units for coordinates and forces	Implement automated unit compatibility checks when connecting FMUs/OpenSim models Standardize unit handling inside the framework Provide clear error/warning messages on mismatched units
UR-3	The framework shall allow the creation of system models by connecting subsystems .	Fulfilled	- Controlled Pendulum Demo: subsystems (Reference, Sensor, PID, Drive, Pendulum) connected into a full system in Modelica Exported subsystems as FMUs and orchestrated connections manually in Python using FMPy - Demonstrated successful data exchange between connected FMUs	- Provide YAML/JSON-based system configuration for easier model assembly Automate connection handling and variable mapping (reduce manual coding) - Extend connection mechanism to handle event-driven interactions
UR-4	The framework should allow the acausal equation-based modeling of the component behavior.	Fulfilled	OpenModelica + OMEdit used for subsystem modeling (Drive, Pendulum, PID, Sensor, Reference) Equations expressed acausally (no manual input–output assignment required) OpenModelica compiler resolves causality automatically (DAE handling, index reduction) - Exported as FMUs for integration in Python	- Document best practices for equation-based subsystem creation (templates, examples) - Add library of reusable Modelica templates for common mechatronic components - Consider hybrid extension: support for acausal + event-driven (discrete) subsystems
UR-5	The framework shall enable the causal equation-based modeling of the component behavior.	Fulfilled	- Some subsystems (e.g., Sensor, PID) already modeled causally with explicit input/output ports - Co-Simulation FMUs (from Modelica, OpenSim) expose I/O interfaces by design - Controlled Pendulum Demo: causal signal flow in the orchestration layer	- Provide clear guidelines/templates for causal subsystem modeling within the framework - Add automated checks for I/O compatibility when connecting causal subsystems
UR-6	The framework shall support the definition of component behavior using discrete-time or hybrid dynamics .	Partially	- Discrete PID controllers implemented in Modelica (sampled when sample(...)) - ImpactWall modeled both as continuous contact dynamics and discrete event (when/reinit) in Modelica - OpenSim Hunt–Crossley contact provides hybrid continuous–discrete behavior (force + event-like contact activation) - Python orchestration loop handles stepwise updates	Extend orchestration with event handling & rollback for hybrid FMUs - Provide examples beyond the pendulum (e.g., switching controllers, digital sensors)

ID	User Requirement	Status	Justification	Further Improvements
UR-7	The framework should provide templates for the creation of equation-based component models .	Partially	- Current subsystem models (Pendulum, Drive, PID, Sensor, Reference) serve as examples but not as reusable templates - No formalized template library exists yet in the framework Users must currently rely on OMEdit + Modelica standard library	Develop template models (e.g., mechanical body, motor, sensor, controller) in Modelica within a dedicated package Provide documentation + guidelines for extending templates Integrate template generation into framework workflow (e.g., pre-packaged FMUs for common components)
UR-8	The framework shall allow the reuse of components and subsystems with different configurations.	Partially	Subsystems in Modelica (e.g., PID, Drive, Sensor) are parameterized and can be reused with different settings FMUs can be instantiated multiple times with different parameter values - Demo models already reused the Sensor for reference and state signals	Provide systematic configuration management (e.g., YAML/JSON config files) - Add support for saving/loading predefined subsystem configurations

ID	User Requirement	Status	Justification	Further Improvements
UR-9	The framework shall enable the integration of subsystem models developed using different modeling tools .	Fulfilled	FMUs exported from OpenModelica integrated via FMPy OpenSim pendulum model integrated alongside FMUs in same co-simulation loop - Demonstrated heterogeneous integration (FMUs + OpenSim + FEM) in Controlled Pendulum demo Model switching and state synchronization (implemented)	Combine models in a MultiComponent instance for interaction with other CoSimComponents
UR-10	The framework should support the combination of equation-based and other heterogeneous sub models .	Fulfilled	Equation-based subsystems modeled in Modelica and exported as FMUs Combined with OpenSim pendulum model in one co-simulation loop Python master orchestrates data exchange consistently	Formalize interface contracts between equation-based FMUs and heterogeneous models (OpenSim, FEM) CoSimComponent as interface implemented

ID	User Requirement	Status	Justification	Further Improvements
UR-11	The framework shall enable the numerical simulation of hybrid systems with both continuous and discrete-time behavior.	Partially	• Discrete PID controllers modeled in Modelica (sampled dynamics) • ImpactWall implemented in two variants: • continuous compliant model (stiffness + damping) • discrete event-based model (when + reinit) • OpenSim Hunt–Crossley contact adds hybrid force law • Co-simulation loop executes continuous + discrete subsystems together	• Implement robust event-handling and rollback in the Python master • Demonstrate multi-event scenarios (e.g., multiple collisions, switching control laws) • Generalize framework support for hybrid FMUs
UR-12	The framework shall allow the user to configure and select solver types and step sizes for simulation.	Partially	• Global master communication step size h is user-set in your Python loop (FMPy) • OpenSim side: integrator accuracy / step can be set via the Manager/Integrator • CS-FMUs bring their internal solvers • step size and tolerances can be influenced via doStep size and via FMU parameters (when exposed).	• Add a uniform config layer (YAML) for: global master step, per-component step sizes, synchronization (fixed/adaptive) • OpenSim integrator type (e.g., Runge–Kutta, implicit) and accuracy/step controls • Implement per-component rate (multi-rate co-simulation)

ID	User Requirement	Status	Justification	Further Improvements
UR-13	The framework should provide a visual representation of the system model and its hierarchical structure .	Fulfilled	- System graph builder implemented - Graph builder uses system connections and involved ports to generate the system graph (implemented)	Export to graph for PNG/SVG and - Support: system → subsystem → component and port-level edges. - Overlay units, rates (sample times), and domains (mechanical/electrical/biomech/FEM) with color coding (implemented)
UR-14	The framework should allow the user to observe simulation progress through graphical and textual output.	Partially	- Python co-simulation loop already prints stepwise logs (t, u, torque, states). - FMPy / FMU APIs provide status + error logs - OpenSim side: print manager/integrator accuracy and states - Added Simulation diagnostic for Pendulum update of FEM model scene	- Add a unified Logger & Progress module (structured logs, tqdm progress bar) - Standardize signal logging to CSV include units - Provide live dashboards (matplotlib animation Plotly) for key traces - Event logs (impacts, saturations, limiter hits, FMU events, rollbacks)
UR-15	The framework should allow the user to compare simulation results across different configurations .	Partially	- Run multiple configs (e.g., PyFMI vs. FMPy, OpenSim vs. Modelica pendulum) and plot traces (q_ref, q_state, torques) side-by-side - FMU/OpenSim runs can be parameterized via Python and re-run to produce comparable outputs.	- Add a RunManager that: accepts a YAML sweep (grid/random search of parameters, solvers, stepsizes), assigns run IDs, and captures config - Standardize results to a common ResultStore with metadata (units, value refs, component)
UR-16	The framework should support result export and post-processing functionality.	Open	- Current notebooks/scripts already collect time series (q_ref, q_state, torques) and plot with matplotlib. - Raw data are available for	- Add a unified ResultStore - Include metadata: run ID, configs, tool versions (FMPy, OpenSim), units, value references, sampling rates

Framework Core Expansion:

- **New Components:**
 - OpenSimComponent: General Component Class for OpenSim models
 - FEMComponent: General Component Class for netgen/NGSolve FEM models
- **Multicomponent Concept:**
 - Combine several representations of the same “thing” (e.g., pendulum FMU + OpenSim + FEM)
 - Implement state synchronization and rule-based switching between models
- **Hybrid Capabilities:**
 - Support discrete and continuous states
 - Introduce hybrid DAE event handling (zero-crossing detection, event-triggered switching)

Simulation Engine & Computational Causality:

- **Goal: Enable robust, adaptive, and event-driven co-simulation**
- **Multi-rate / Adaptive Time Stepping:** Each component advances with its local solver time base, synchronized at communication points
- **Event-Handling:** Detect contact events, synchronize discrete and continuous subsystems
- **Causality Solver:**
 - Extend execution order computation to handle cyclic dependencies
 - Build incidence matrix and apply Pantelides algorithm for algebraic loop resolution
- **Result Handling:** Standardize simulation metadata and result exports for traceability

User Interaction & System Assembly

- **Goal: Provide an intuitive, guided workflow inside Jupyter**
- **Multi-Step Wizard Interface (Standard & Expert Mode)**
 - Model selection -> FMU Export -> system assembly -> configuration -> simulation
 - Live preview of Modelica code
- **Configuration Management:**
 - Save and reload full system configurations (components, connections, parameters)
 - Implement configuration validation
- **Visualization:**
 - Generate system graphs
 - Enable live plotting and simulation diagnostics for monitoring

Framework Integration & Validation

- **Goal: Consolidate core modules and ensure framework stability**
- **Testing and Automation:**
 - Update and expand unit tests for all framework layers
 - Unit tests for core classes, wrappers, and port connections
 - Integration tests for system assembly and simulated run
 - Cross-platform validation (Windows/Linux/Mac)
- **Data Layer:** Structured result logging for reproducible simulation data
- **Documentation and Packaging:**
 - Refactor into modular pyproject-based package
 - Add tutorials, notebook, and licensing for distribution